TECHNICAL ASSESSMENT

Build a Talent Market Scout Agent

A human-in-the-loop AI agent, built with LangGraph and LangChain, that turns a hiring request into an evidence-backed talent market brief: how many jobs are open, what they pay, which skills employers want, and how hard the role will be to hire.

DURATION	MODE	DELIVERABLES	DEADLINE
Take-home, 5–7 days	**Individual project**	**GitHub repo, write-up, live walkthrough**	**Monday, 2 November 2026, 9:00 AM** Extensions on request; ask in advance

OVERVIEW

Background

Joveo helps employers and staffing firms hire faster by placing and optimising job ads across hundreds of publishers. Every campaign starts with the same questions from a client: how competitive is this role in this location, what should we pay, and which skills should the job description lead with.

Today a customer success manager answers these by hand: searching job boards, eyeballing salary ranges, and pasting findings into a deck. It takes hours per role, the quality varies by person, and the answer is rarely backed by traceable evidence.

AI agents can do this research loop well, but only if they know when to ask a human for clarification, when the data is good enough, and when to stop. This assessment asks you to build exactly that: a **Talent Market Scout Agent** that takes a hiring request, clarifies it with the user, gathers real job postings, and produces a structured market brief a CSM could hand to a client.

OBJECTIVE

What We Are Evaluating

This assessment evaluates your ability to:

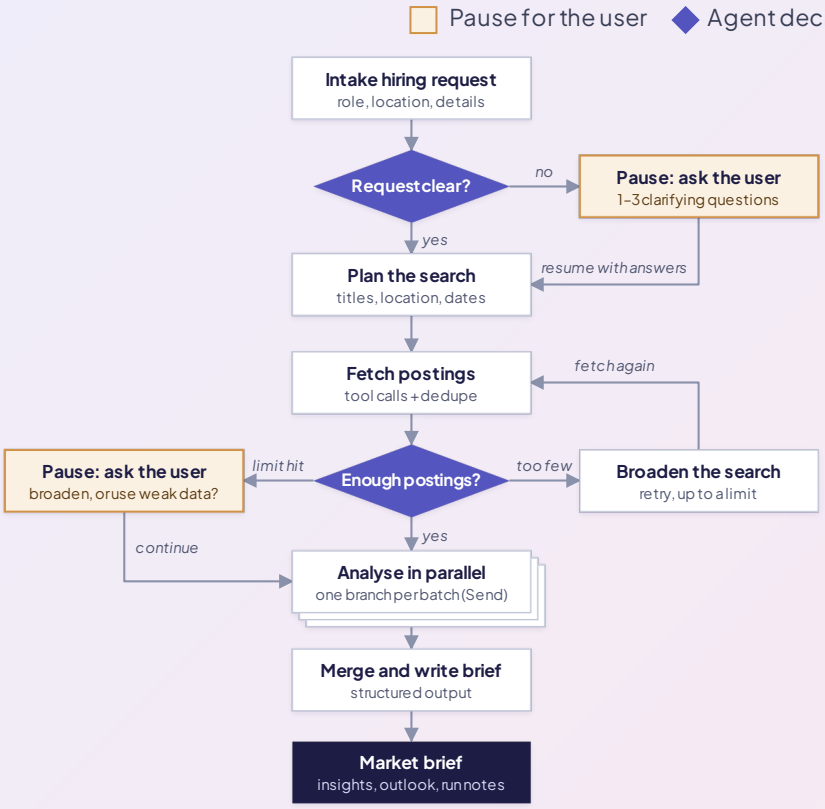
- Design a stateful, multi-step agent as an explicit graph in LangGraph, with clear state, nodes, and routing decisions
- Build human-in-the-loop checkpoints where the agent pauses, asks the user for what it needs, and resumes correctly
- Write reliable prompts and structured outputs for a production-style task
- Integrate with a real-world data API and an LLM provider, and use tool calling so the model decides which tool to use
- Handle large result sets, sparse or messy data, ambiguous inputs, and API failures without crashing
- Explain your design decisions clearly in writing and live in the interview

AGENT SPECIFICATION

What the Agent Should Do

Given a hiring request as input, your agent must complete the steps below. The order is a guide; the routing between steps (loops, pauses, early exits) is yours to design.

Reference flow



A reference shape, not a required design. The low-data pause on the left is a good-to-have.

1 STEP 1 — Intake the request

Accept a hiring request with: role or job title, location, and any optional details the user gives (seniority, employment type, remote/hybrid, currency, the client's purpose). Store everything in a typed graph state.

2 STEP 2 · HUMAN-IN-THE-LOOP — Clarify with the user

Before doing any research, the agent checks whether the request is specific enough. If it is not, the graph must **pause at a node**, show the user 1–3 targeted questions, wait for answers, and then **resume from the same point** with the answers added to state.

Examples: "Engineer" is ambiguous (software, civil, mechanical?). "Texas" may mean the whole state or one metro area. "Senior" may mean 5+ years or a specific level. If the request is already clear, the agent should skip this pause rather than ask questions for the sake of it.

3 STEP 3 — Plan the search

The agent drafts a search plan: title variants and synonyms, location scope and radius, date range, and the target number of postings. The plan is stored in state so later steps and the write-up can refer to it.

4 STEP 4 Gather postings

Use tool calling to fetch real job postings from a public jobs API, then deduplicate them. After each fetch, the agent decides whether the sample is good enough:

- **Enough relevant postings** → continue to analysis
- **Too few or too noisy** → broaden or refine the search and fetch again, within a fixed limit of attempts
- **Still too few after the limit** → pause and ask the user whether to broaden further or continue with a weak sample

5 STEP 5 Analyse the market

Pass the postings to an LLM and analyse them across the dimensions below. For large result sets, split postings into batches, analyse batches in parallel, and merge the results.

Dimension	What to extract or compute
Demand & volume	Number of relevant active postings, how recent they are, and any trend visible in posting dates
Compensation	Salary range (median, middle 50%), share of postings that disclose pay, normalised to one currency and period
Skills & requirements	Most-requested skills (normalised so "Postgres" and "PostgreSQL" count once), must-have vs nice-to-have, typical years of experience
Competition	Top hiring employers, and the split between direct employers and staffing agencies
Titles & seniority	The title variants employers actually use, and the seniority mix

6 STEP 6 Generate a structured market brief

The agent must output a structured brief (JSON and a readable Markdown rendering) containing:

- **Summary** — 2–3 sentences on what the market looks like for this role and location
- **Insights** — a list of findings, each tagged with a confidence level and backed by evidence (the count and IDs of the postings it is based on)
- **Client recommendations** — a suggested job title, salary band, and the top skills to lead the job description with, each tied to evidence
- **Hiring outlook** — one of Favourable / Competitive / Hard to fill, with a one-line reason
- **Run notes** — questions asked and answers given, the final search plan, postings analysed, and any assumptions made

Confidence tags

On every insight; you set and justify the thresholds

High	Strong, consistent evidence across many postings; safe to share
Medium	Partial or mixed evidence; share with a caveat
Low	Thin evidence; directional only

Hiring outlook

Exactly one per brief, with a one-line reason

Favourable	Plenty of talent relative to demand; standard pay is competitive
Competitive	Many employers chasing the same profile; pay and ad quality matter
Hard to fill	Scarce talent or high demand; expect long time-to-hire or premium pay

STEP 7
7 Handle edge cases gracefully

Your agent should not crash or return garbage when faced with:

- Very large result sets (more than 500 postings): implement a batching strategy so you stay within the LLM's context window
- Niche roles or small locations with very few postings
- Ambiguous or incomplete requests, including a user who gives a vague answer to a clarifying question
- Missing or inconsistent salary data (hourly vs annual, mixed currencies, no salary at all)
- Duplicate postings of the same job across locations or sources
- Jobs API rate limits, timeouts, and transient errors
- A user who stops at a pause and comes back later: the run must resume from where it stopped

REQUIREMENTS
Technical Requirements

Minimum bar — MUST HAVE

- Built as a LangGraph `StateGraph` with a typed state schema (TypedDict or Pydantic); you can explain every field
- At least one human-in-the-loop pause using LangGraph's `interrupt()`, resumed with `Command(resume=...)` (the Step 2 clarifying questions)
- A checkpointer (for example `MemorySaver` or `SqliteSaver`) so a paused run can be resumed on the same thread
- Conditional edges for the agent's routing decisions (clarify or skip, enough data or fetch again)
- Tool calling: the jobs API exposed as LangChain tools (`@tool`), bound to the model, and run through a `ToolNode` or equivalent
- Structured output: the market brief produced through Pydantic models (for example `with_structured_output()`), not parsed from free text
- Fetches real job postings from a public jobs API and runs end-to-end on at least one real role and location
- A clear README with setup and run instructions

Good to have — RECOMMENDED

- Parallel batch analysis using LangGraph's `Send` API (map-reduce), with a reducer that merges batch results in state
- A second pause: search-plan approval, where the user can approve, edit, or reject the plan before any fetching
- A third pause: the low-data checkpoint described in Step 4
- Node-level retry policies for API calls, and model fallbacks (`.with_fallbacks()`) when the LLM fails or is rate-limited
- Recovery when the LLM returns malformed output
- A simple CLI or minimal web UI (Streamlit, Gradio, or plain HTML) that handles the pauses properly
- Configuration flags, for example `--focus salary` or `--focus skills`

Stretch goals — BONUS POINTS

- **Supervisor multi-agent setup:** a supervisor that delegates to specialist agents (search, skills analyst, salary analyst, report writer)
- **Critic pass:** a reflection node that reviews the draft brief for claims not backed by postings and sends it back for revision
- **Streaming progress:** node-by-node progress shown to the user as the agent works ("Fetching… 142 postings found… analysing batch 3 of 6…")
- **Long-term memory:** a LangGraph `Store` that remembers user preferences across runs (currency, default seniority, skills to always track)

Language, frameworks, and data

Language: Python is strongly preferred; any language is fine if clearly justified.

Frameworks: LangGraph and LangChain are required. You may add other libraries, but must be able to explain every component.

LLM API: any provider (Claude, GPT, or Gemini). Keep your API key in a `.env` file; never commit it.

Jobs data: the Adzuna API (free key; India, US, UK and more) or USAJobs. Another public jobs API is fine if justified; no scraping sites whose terms forbid it.

DELIVERABLES

What to Submit

1 — GitHub repository

- Clean, well-structured code with meaningful commit messages
- A `README.md` covering what it does, setup steps, how to run it (including how to answer the pauses), and known limitations
- A `.env.example` file listing required environment variables
- No hardcoded API keys or secrets anywhere in the repo
- The repository must be public so we can clone and run it

2 — Three sample runs

Run your agent on three real hiring requests of your choice and include the outputs in a `samples/` folder. Each sample should include the final brief (JSON and Markdown), the full transcript of any questions the agent asked and the answers given, and a trace or log of the agent's routing decisions. Aim for variety:

- **One clear, common request** (for example "Data Analyst, Bangalore") that goes straight through with no clarifying questions
- **One ambiguous request** (for example "Engineer, Texas") that triggers the clarifying-question pause
- **One high-volume or low-volume request** that stresses your batching logic or your sufficiency loop

Tip — pick requests wisely. Choose roles and locations where the jobs API returns real postings, so your agent has enough evidence to say something meaningful.

3 — Written write-up (1–2 pages)

A short document (Google Doc, Markdown, or PDF) covering:

- **Architecture** — your graph (include a diagram; LangGraph can export one), your state schema, and why you structured it this way
- **Human-in-the-loop design** — where the agent pauses, why there and not elsewhere, and how resuming works
- **Prompting strategy** — how you designed your prompts and structured outputs, and what you changed after testing
- **Large and small result sets** — how you handled context window limits and how the agent decides the data is enough
- **Failure modes** — what the agent gets wrong and why (be honest)
- **What's next** — what you would build or improve with more time

EVALUATION

How We Evaluate

Area	Weight	What we look for
Graph architecture & state design	**25%**	Clear state schema, sensible nodes and edges, routing decisions made in the graph rather than hidden inside one big node
Human-in-the-loop & robustness	**20%**	Pauses happen at the right moments, resume correctly, and edge cases are handled without crashing
End-to-end output quality	**20%**	Briefs are accurate, specific, and every number traces back to real postings
Prompting & structured output	**10%**	Reliable prompts, well-designed output schemas, recovery from bad output
Code quality & README	**10%**	Readable, well-organised code; someone else can set it up and run it
Stretch goals	**10%**	Each well-integrated stretch goal adds here
Write-up	**5%**	Clear reasoning and honest self-assessment

Live walkthrough. In the follow-up interview you will share your screen and run the agent on a role and location we give you on the spot, explain its decisions from the trace, and make one small change to the graph live (about 20–30 minutes).

RULES

Constraints & Rules

- Do NOT use an off-the-shelf labour-market or salary-intelligence product as the core of your solution; the point is to build the agent logic yourself
- LangGraph and LangChain are required; prebuilt agent shortcuts (for example `create_react_agent`) may be used for a sub-part only if you can explain what they do inside and why you chose them
- Use any LLM provider you prefer; you are responsible for any API costs
- Do NOT submit a solution that is entirely AI-generated with no meaningful human authorship; the live walkthrough will make this clear
- Keep your GitHub repo public so we can clone and run it

HOW TO SUBMIT

Submission

When you are ready, send us the following before the deadline on the first page:

- The link to your public GitHub repository
- The link to your written write-up (Google Doc, Notion, or attached PDF)

There is no single correct answer. The goal is to show clear thinking, sound engineering judgement, and honest self-awareness about where your solution falls short. Good luck!